RISC-V INTERNATIONAL RFP · TECHNICAL PLAN

Automatic Test Generation from the Sail Golden Model

We generate RISC-V tests from the upstream Sail model, targeting the Privileged Architecture, and wire the result into the Golden Model's own CI.

---

1 · OBJECTIVE AND DELIVERABLES

Objective

We take one Golden Model configuration and the model itself. We produce standalone RISC-V executables, organised by extension, plus the evidence needed to judge what they cover.

We read the instruction set from the model's own declarations. The suite therefore tracks the model, not a list we maintain by hand.

#	DELIVERABLE
1	Repository with framework, user-facing and developer documentation
2	Example scripts generating a suite from a given configuration
3	Example scripts executing a suite, collecting results, summarising
4	CI integration into the Golden Model
5	Required Golden Model changes

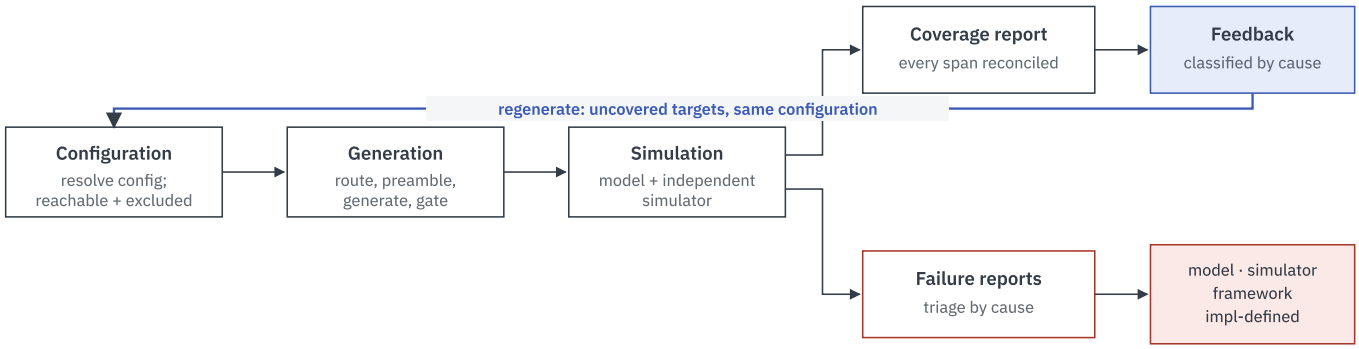
2 · METHODOLOGY

How the framework works

Six subsections: the pipeline and what it measures, how generation is routed between two backends, how the privileged architecture is reached, how a test is proven able to fail, what the suite looks like, and what runs in CI.

2.1 Sail model to coverage report

Six stages in a closed loop. Coverage is not only a report — it is the feedback that drives generation of tests for what is still missing. Each stage carries a distinct responsibility.



The whole pipeline runs separately for each of the seven configurations, and each reports its own coverage rather than a combined figure.

Coverage here is Sail specification branch coverage, measured by the model's own instrumentation rather than by a proxy. Building with `-DCOVERAGE=ON` passes `--c-coverage` to the Sail compiler, which emits a `.branch_info`

manifest of every branch the source declares, and defines `SAILCOV` in the emulator, which appends to `sail_coverage` as each span executes. The repository documents this as specification coverage measurement in testing.

Coverage and failure reports are parallel outputs of simulation, not sequential: coverage comes from that instrumented run, failure reports from comparing verdicts. A failing test writes no coverage data, so the two paths carry different tests.

STAGE	WHY IT IS NECESSARY
Configuration	Prevents inconsistent configuration and removes unreachable targets from the denominator.
Generation	Routes each target to an appropriate generator and produces tests that can detect failures.
Simulation	Provides Sail reference execution and an independent check of the same ELF.
Coverage Report	Separately measures reachable-target coverage, Sail source coverage, and test sensitivity per configuration.
Failure Reports	Turns disagreements into actionable model, simulator, framework, or implementation-defined findings.
Feedback & Record	Classifies uncovered targets and feeds them back into generation to close the coverage gap.

Coverage: what we measure

We measure four things and report them separately. One number cannot express completeness, and these four are not interchangeable.

MEASURE	QUESTION IT ANSWERS
Reachable coverage	Does every reachable target have a test?
Model source coverage	Which Sail source branches were exercised?
Fault sensitivity	Can each test fail — proven by a passing and a deliberately broken run?
Differential agreement	Does a simulator not derived from the model agree?

The committed configurations

> These seven configurations come from the Golden Model's own test suite. They are not a free XLEN × VLEN × ELEN cross-product; the model constrains which combinations are valid.

CONFIGURATION	XLEN	VLEN	ELEN	GENERATION ROUTE
`rv32d_v64_e32`	32	64	32	Symbolic
`rv64d_v64_e64`	64	64	64	Symbolic
`rv32d_v128_e32`	32	128	32	Symbolic
`rv64d_v128_e64`	64	128	64	Symbolic
`rv32d_v256_e32`	32	256	32	Oracle only
`rv64d_v256_e64`	64	256	64	Oracle only
`rv64d_v512_e64`	64	512	64	Oracle only

> **Each configuration has its own reachable target list, exclusion list and coverage result. Results are reported separately, not averaged. VLEN=256 and VLEN=512 use the concrete oracle because they exceed the symbolic engine's 129-bit value limit.**

2.2 ISLA vs Oracle

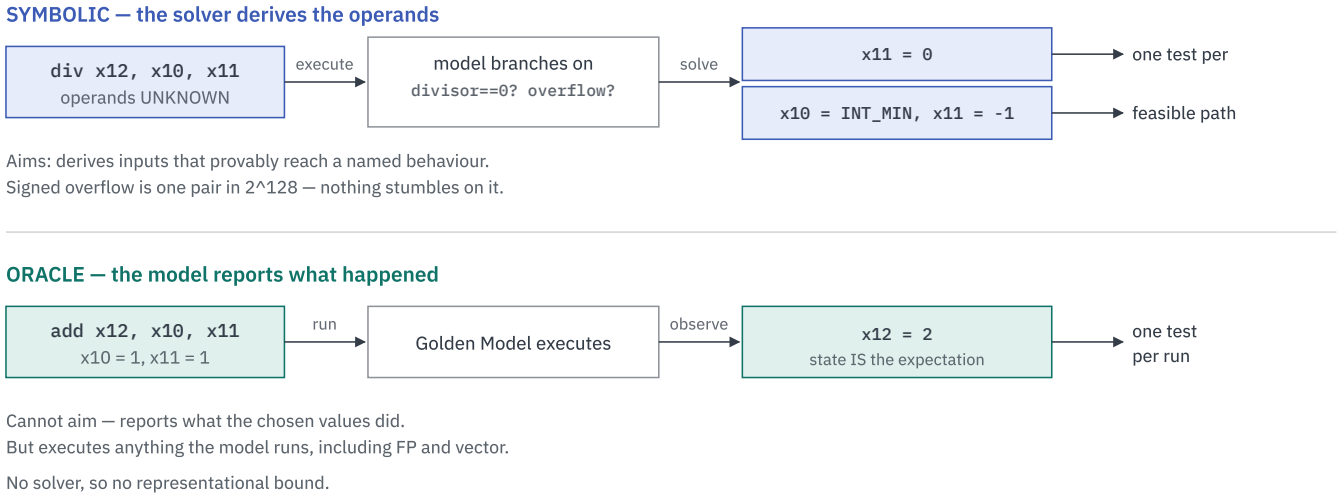
Three families of generator are relevant. Each works. Each fails on a different part of this problem — which is why we route between them rather than pick one.

APPROACH	REPRESENTED BY	WHAT IT DOES	WHY IT IS NOT SUFFICIENT ALONE
Constrained random	the riscv-dv lineage	Generates instruction streams under architectural constraints	Cannot aim at a named behaviour in the model, and still needs a separate oracle for expected results
Concrete oracle	`rems-project/sail-riscv-test-generation`	Chooses instructions and operands, runs the model, records the resulting state as the expectation	Cannot construct privileged machine state, and cannot aim — it reports what the chosen values did
Symbolic path enumeration	`rems-project/isla-testgen`	Leaves operands unknown and solves for values reaching each feasible path	Cannot execute floating point or vector; value representation is bounded at 129 bits

THE METHODOLOGY ADOPTED

We use both: the concrete oracle and symbolic path enumeration, as **two backends under one control loop**, with a small template backstop. Their limitations are opposites. Privileged behaviour needs the machine in a specific state *and* an instruction executed there, and neither approach gives us both.

ISLA and the oracle, on one instruction



Where the oracle is blind, the solver aims: the model's divide has a signed-overflow case requiring the dividend to be exactly the most negative value and the divisor exactly `-1` — one pair in 2^{128} at 64-bit width, which no value-choosing generator will land on. Where the solver stops, the oracle continues: floating point and vector together are more than half the model's instruction declarations, and the oracle runs both unmodified.

Generation backends

BACKEND	WHAT IT IS FOR
ISLA + Z3	Derives operands that provably reach a named behaviour; solves for privileged machine state
Concrete oracle	Executes anything the model can run, including FP and vector. No solver, so no solver limits
Templates	A hand-written backstop for the few targets **neither engine can produce a valid test for**. Capped at 4 builders; growth is a defect signal, reported in the suite summary rather than counted as progress

Why a third backend exists at all. Control-flow instructions defeat both engines, for unrelated reasons. The symbolic engine does not model a compressed instruction's control-flow effect, so `c.jr` and `c.jalr` emerge with nothing constraining the jump target — the test jumps wherever the register happened to point. The oracle cannot cover them either, because a jump's target address differs between the probe run and the final linked ELF, so the recorded expectation would not match what executes. A hand-written template solves both by making the observable result independent of the address: jump to a local label, place an instruction between the jump and the label that would corrupt a checked register if it ever ran, then clear the address-holding register. That is what makes it a test of the jump rather than of nothing.

It is deliberately a backstop, not a general-purpose tool. Every template is a target the generators could not reach, so the count is reported with the suite: **if it grows, that is evidence of an engine gap to be fixed, not of coverage gained.**

Generation routing

We route each target with a table, not hard-coded dispatch. Most of the ISA is reachable by either engine, so the choice is real. Where it is not, we record the representational limit next to the routing decision. A new extension later is one table row, not a new engine.

ISA SURFACE	SYMBOLIC	ORACLE	ROUTED TO
I, M, A, B, C, Zicsr, `Zb*` / `Zc*` / `Za*` / `Zk*`	Yes	Yes	Symbolic, oracle as fallback
CSR-only privileged extensions `Sstc`, `Sscofpmf`, state-enable, pointer masking	Where expressible as constraints	Yes	Either
F, D, `Zf*` / `Zd*` / `Zh*`	**No** — 45 soft-float primitives absent from the engine	Yes	Oracle only
V, `Zv*`, vector crypto	**No** — vector length is runtime state; value representation bounded at 129 bits	Yes	Oracle only
Branch, jump	Partial	Partial	Template — capped at 4

What each target type needs from a generator

TARGET TYPE	WHAT IT COVERS	BEST SUITED FOR
Instruction sequences	ALU ops, loads and stores	Functional execution and corner cases
CSRs / state	`Zicsr`, counters, state enable, privilege state	CSR access, privilege transitions, machine-state scenarios
Traps / exceptions	Illegal instruction, page faults, `ecall` / `ebreak`	Exception handling, trap entry and return, fault scenarios
Interrupts	Timer, external, software	Delivery, priority, delegation
Translation / memory	Page tables, Sv modes, PMP/PMA	Virtual memory, access faults, protection
Privileged features	M/S/U modes, counters, state-enable, QoS	Mode transitions, delegation, feature enablement

Why the proposed framework differs from the existing generator

The existing REMS generator provides useful prior art, but this proposal requires additional capabilities for configuration-aware privileged-ISA coverage and model evolution. The framework therefore builds on the same ecosystem while extending the generation flow in the areas that matter to this RFQ.

AREA	EXISTING GENERATOR	PROPOSED FRAMEWORK
Instruction source	Uses a fixed generator-side instruction description	Derives the instruction inventory from the Sail model at generation time
Encoding	Generator-defined	Assembly is emitted and encoded by the target toolchain
Configuration	General ISA generation	Filters targets against the selected model configuration before generation
Privileged state	Not the primary generation mechanism	Builds required machine state through the preamble
Generation method	Primarily concrete generation	Routes targets between symbolic, concrete, and template backends
Validation	Test generation / model execution	Fail-capable tests plus differential execution on an independent simulator
Model evolution	Requires generator-side updates when its source assumptions change	Model-derived targets allow new Sail functionality to enter the generation flow without rebuilding the instruction inventory by hand

> The key design choice is to make the Sail model the source of truth for what the framework can generate, while keeping generation, privileged-state construction, validation, and reporting as replaceable components. This allows the framework to follow model evolution without turning each new Sail feature into a separate generator change.

2.3 Privileged architecture

The model implements the whole privileged architecture through **nine instructions**: `ECALL`, `EBREAK`, `MRET`, `SRET`, `WFI`, `SFENCE.VMA`, and `Svinval`'s `SINVAL.VMA`, `SFENCE.W.INVAL` and `SFENCE.INVAL.IR`. Count only those meaningful exclusively in a privileged context and it is seven. So we get coverage from machine state we build in the preamble, not from picking opcodes. The preamble builder, not the instruction generator, is the load-bearing part.

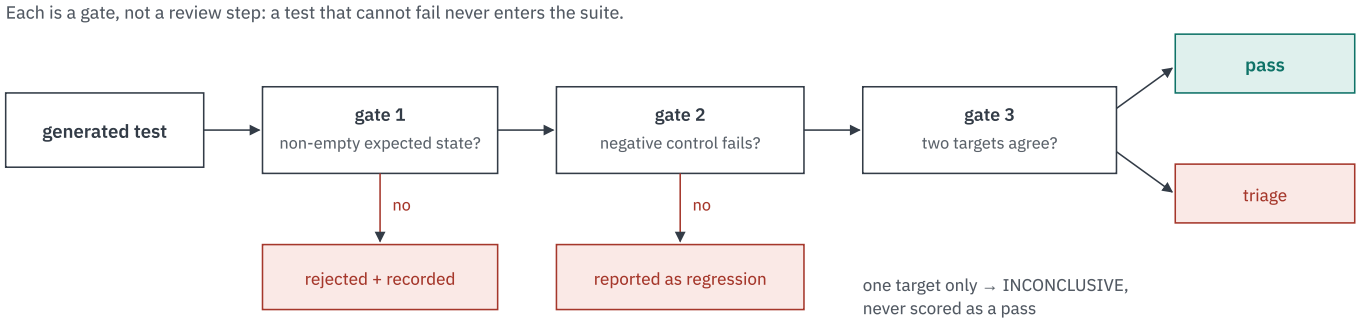
SURFACE	MECHANISM	BECOMES A TARGET AS
M / S / U modes	Preamble sets `mstatus.MPP` / `SPP`; returns via `MRET` / `SRET`	one per mode per behaviour
CSRs	Model-derived register set swept per instruction, at each legal privilege	one per (CSR, access, privilege)
Traps & exceptions	Preamble constructs the faulting condition; handler asserts both cause and origin	one per exception cause
Interrupts & delegation	Pending and enable bits set directly; `mideleg` toggled	one per cause × {taken, not taken, delegated}
Virtual memory	Preamble builds a page table and points `satp` at it	one per mode × PTE feature × fault class
PMP	Preamble programs entries with the correct order and lock bits	one per match type × access × privilege
PMA	Region attributes come from the model's configuration JSON	one per attribute, under a config declaring the access illegal

We make interrupts deterministic in the preamble rather than waiting for one. A self-checking ELF cannot wait for a timer across four targets with four notions of time. So we set the pending bit, the enable, and where needed the global enable directly. The assertion then checks *which mode* the trap landed in, not merely that one occurred.

We exercise PMA through the configuration, not an instruction. Physical memory attributes are platform-defined region properties, not state a preamble can write. So a PMA test is an ordinary access, run under a configuration whose region declares that access illegal. That makes the negative control exact: the same ELF under the stock configuration must pass.

2.4 Validation

A test that cannot fail still executes, and still raises the coverage figure. We stop that with three gates. Each is a gate, not a review step — nothing reaches the suite by inspection.



Gate 1 answers “can this test fail at all”; gate 2, “can it tell a correct trap from one that never fired”; gate 3, “is this more than the model agreeing with itself”.

MECHANISM	WHAT IT REJECTS
Validation gate	Any test with an empty expected result. The rejection is recorded with its cause, never silently dropped
Negative controls	Every trap-asserting scenario ships with the same setup minus the triggering condition. It must fail. A control that stops failing is reported as a regression
Differential execution	Agreement with the model alone. Where only the model is available the result is reported inconclusive, never scored as a pass

> **Two invariants govern a trap-asserting test, and they are independent.** The handler must check the *cause*, so a trap of the wrong kind fails rather than passing as “a trap occurred”. It must also check the *origin*: the trap vector is installed by the preamble's first instruction, so every later preamble step runs with the handler live, and a fault raised there carrying the expected cause would otherwise report success having never reached the instruction under test. The negative controls do not cover that case, because they vary the expected cause rather than the trap's origin.

2.5 Output

One directory per configuration, one subdirectory per extension, with a manifest carrying each test's extension, target, originating backend, required configuration predicates, whether it is a negative control, and a provenance stamp. Selection is a predicate over that manifest rather than a naming convention, so selection and coverage accounting cannot disagree.

	SELF-CHECKING PROFILE	ARCH-TEST PROFILE
Pass/fail via	HTIF `tohost` write	Signature region, dumped and compared

	SELF-CHECKING PROFILE	ARCH-TEST PROFILE
Code placement	Preamble at the target's fixed boot address	Test bytes at `0x80000000`
Entry symbol	Framework's own preamble	`rvtest_entry_point`
Consumers	Sail, Spike, QEMU, RTL	The arch-test pipeline

These cannot be one ELF, and that is the consumers' constraint rather than ours. arch-test's linker script wants test bytes at `0x80000000`; fixed-boot-address mode wants the preamble there. One address, two claimants. So a generation run targets one consumer or the other.

2.6 Continuous integration

JOB	TRIGGER	DOES	BUDGET
Suite replay	Every PR	Runs the checked-in suite against the model; reports pass/fail with the triage split	Minutes
Regenerate and re-measure	Scheduled	Regenerates from the current model, re-runs, re-measures coverage, reports drift against the baseline	Off the PR path

Generation is solver-bound and can take longer than a per-PR check should. Replaying a prebuilt suite is fast. We split them so two different signals do not arrive as the same red mark.

A failing PR job means the change under review altered behaviour the suite had pinned. A failing scheduled job may instead mean the suite needs regenerating against a changed model.

3 · SCOPE

What is committed, deferred and excluded

AREA	COMMITTED	DEFERRED — ARCHITECTURE READY	EXCLUDED
M-mode	CSR access, trap entry and return, privilege transitions, PMP, PMA, interrupt delivery and delegation	Nested traps	—
S-mode	All translation schemes — Sv32, Sv39, Sv48, Sv57, Svbare — with negative controls, plus PTE content features and the fault taxonomy	—	—
Privileged extensions and capabilities	29 of 29 — 27 extensions plus the S and U privilege modes, which the model declares as `Ext_S` / `Ext_U` because they are optional	—	—
Unprivileged ISA	Only what privileged scenarios require	Systematic generation across the unprivileged ISA	—
Floating point	—	F/D instruction functionality. Configurations with F and D enabled are still generated and run	—
Vector	—	Vector instruction functionality, same reason and same route	—
Hypervisor	—	—	Not implemented in the model

The 29 privileged targets, enumerated

Two of these are privilege modes rather than extensions. The model declares them as `Ext_S` / `Ext_U` because that enum encodes what a configuration can turn off, and a hart may be M-mode-only. M-mode has no such clause — it is not optional — so it is committed in its own row above rather than counted here.

GROUP	TARGETS	N	HOW WE REACH IT
Privilege modes not extensions	`S`, `U`	2	Enter and transition between privilege modes
CSR access and counters	`Zicsr`, `Zicntr`, `Zihpm`, `Smcntrpmf`, `Sscounterenw`	5	CSR accesses at each applicable privilege; RV32 adds the high-half registers
State enable, QoS, overflow	`Smstateen`, `Ssstateen`, `Ssqosid`, `Sscofpmf`	4	CSR accesses at each applicable privilege
Timer and trap value	`Sstc`, `Sstvala`	2	Configure timer and trap state, then observe the behaviour
Pointer masking	`Smmpm`, `Smnpm`, `Ssnpm`	3	Run existing instructions with masking enabled — no instructions of its own
Translation modes	`Svbare`, `Sv32` (RV32), `Sv39`, `Sv48`, `Sv57` (RV64)	5	Configure translation, build page tables, execute memory accesses
Translation maintenance	`Svinval`	1	Execute the relevant instructions — the only group that has any
Page-table entry content	`Svade`, `Svadu`, `Svnapot` (RV64), `Svpbmt` (RV64), `Svrsw60t59b`	5	Construct the relevant PTEs and exercise translation
Translation guarantees	`Svvptc`, `Ssccptr`	2	Observe a property of the walk itself, not a dedicated instruction
Total		**29**	

> **Only one group contributes instructions.** Everything else is reached through CSRs, machine state, page tables or configuration — which is why the preamble builder carries this scope, not the instruction generator. XLEN applicability comes from the model: `core/extensions.sail` gates `Sv32` on a 32-bit register width, and `Sv39`, `Sv48`, `Sv57`, `Svnapot` and `Svpbmt` on a 64-bit one. A target that cannot apply to a configuration leaves that configuration's list with that reason recorded — it is not a gap.

We exclude a target for one of two reasons only: the model does not implement it, or no test that can fail could observe it. We record which one, and either is open to challenge.

We exclude Hypervisor because the model's hypervisor file has an enablement clause and no behaviour. There is nothing to target. Enabling it means implementing it upstream, which is model development and costed separately.

4 · IMPLEMENTATION AND SCHEDULE

How the framework will be built, and in what order

We work across five repositories, three of them forks. Throughout, we make existing tools reach the RISC-V model rather than replace them. That keeps the engineering surface small and the upstream story simple.

The work splits into five workstreams. The first three must land together before we can generate a single symbolic test.

REPOSITORY	ORIGIN	ROLE	WHY IT MUST CHANGE	WS
sail-riscv	fork of `riscv/sail-riscv` BSD-2-Clause	The Golden Model — the specification under test, and the source of every target	Exposes no entry point with the two-phase init/step convention the generator needs. Adding one, plus a workaround for a Sail codegen limitation, is **Deliverable 5**	B
isla isla-lib, isla-sail	fork of `rems-project/isla` BSD-2-Clause	Symbolic execution library, and the Sail-to-IR compiler it depends on	Lacks primitives this model calls, assumes 64-bit addresses, and cannot represent struct-typed vector elements — without which PMP cannot be tested symbolically at all	A
isla-testgen	fork of the CHERI/Morello generator BSD-2-Clause	Turns symbolic execution into materialised ELF tests	Has no RISC-V target, and carries three assumptions true only of the architectures it already supports	C
Concrete oracle	written new Apache-2.0	Runs the model and records the resulting architectural state as the expectation	No existing tool does this against a configuration-driven model. Depends on none of the above	D
Framework repository	written new Apache-2.0	Orchestration, target enumeration, sweeps, coverage, reporting	The engagement's own deliverable — **Deliverable 1**, and the home of Deliverables 2 and 3	E

Only one of these changes is a committed deliverable. The Golden Model changes are Deliverable 5 and are submitted upstream, so their acceptance is contractual and their merge is tracked as Phase 12. The isla and isla-testgen changes are carried in forks: the RISC-V target is specific to this work and belongs there, while the fixes that are not RISC-V-specific — the address-width assumption, the struct-typed vector elements — will be offered upstream as a matter of good practice. That offer is not committed as a deliverable, because this project cannot commit to another project's merge decision. The two new repositories are ours to deliver outright.

Why this section is specific. A plan that says “integrate the symbolic engine” is not a plan. The items below are the work that integration actually consists of, identified by scoping each tool against the current model. Each names the file or subsystem it lands in, so the estimate in section 4 rests on enumerated work rather than on an allowance.

4.1 Prepare the symbolic execution library

Workstream A — symbolic library `fork of rems-project/isla`

- **Track the Sail release the model compiles with.** The Sail plugin consumes Sail's own Jib IR, whose type definitions and rewrite-pass ordering change between releases. The plugin will be updated against the Sail version pinned at project start, and re-checked whenever that pin moves.
- **Implement the primitives the model calls and the engine lacks.** Scoping against the current model identifies the reservation-set operations (`load_reservation`, `match_reservation`, `valid_reservation`, `cancel_reservation`), `count_trailing_zeros`, `vector_init`, and `sys_enable_experimental_extensions`. Each is a small, self-contained addition; the risk is discovering a further one late, which is why Phase 4's acceptance test is a clean symbolic run over every symbolic-routed extension rather than over a sample.
- **Remove the 64-bit address assumption.** The memory model's overlap check uses a fixed 64-bit address width. RV32 addresses are 32 bits, so the width will be taken from the address's own solver-tracked value.

- **Support struct-typed vector elements.** This is the item PMP depends on. The model represents PMP configuration as `vector(64, Pmpcfg_ent)` — a vector of bitfield *structs*, not of bitvectors — and the engine's value encoding has no case for struct-typed elements, so any symbolic access to a PMP register fails. Vector read and update will construct a per-field conditional instead. Without this there is no symbolic PMP testing, and PMP is a required item in the RFP.

4.2 Propose the model-side entry points

Workstream B — Golden Model `Deliverable 5, submitted upstream`

- **Two entry points.** The generator needs a two-phase init/step calling convention that the model does not currently expose. `isla_testgen_init` and `isla_testgen_step` will be added to `main.sail`, modelled on the existing `isla_footprint` entry points, and preserved against dead-code elimination. A post-build check will confirm they survived compilation.
- **Work around a compiler limitation, minimally.** Sail functions that destructure a parameter of two or more tuple elements currently produce inconsistent synthetic names in the IR. The change is mechanical — destructure in the function body rather than the parameter list — and affects on the order of fifty functions. It alters no architectural behaviour.
- **Consolidate four CSR accessors.** `read_CSR`, `write_CSR`, `is_CSR_accessible` and `is_CSR_exception_virtual` hit the same limitation in the scattered-function merging step rather than in any clause body, so those four will be de-scattered into one function each, preserving first-match-wins clause order. This is the largest single diff of the workstream and will be proposed as its own commit so it can be reviewed on its merits.
- **Make PMP reset concrete.** `reset_pmp()` will build a fully concrete entry per `pmpcfg` register rather than updating the prior symbolic value.

> **These changes are proposed, not assumed.** They will be submitted upstream and are complete only when merged. If the entry points are rejected, the framework falls back to oracle-only generation — a capability reduction rather than a project failure, with privileged scenarios that relied on solver-derived state moving to preamble construction. That fallback is a designed path, not a contingency invented at the time.

4.3 Implement the RISC-V generator target

Workstream C — test generator `fork of the CHERI/Morello isla-testgen`

- **A RISC-V target implementation.** The generator is organised around a target interface of roughly thirty methods covering register naming, PC alignment, entry points and ELF emission. Implementing it for RISC-V is the bulk of this workstream.
- **An ELF emitter** producing standalone, self-checking executables in the two output profiles of section 2.5.

- **Three portability assumptions to remove**, each true only of the architectures the generator already supports:
    - *Boot PC.* The engine sets the initial PC by asserting equality at the solver level, which silently does nothing when the PC is already concrete — as it is for RISC-V, whose init calls `reset()`. The value will be written directly.
    - *PC width against address width.* RV32's PC is 32 bits while the model's memory interface uses 64-bit addresses regardless of XLEN, so the PC will be zero-extended for the fetch.
    - *Register numbering.* General-purpose registers are assumed to start at zero. `x0` is hardwired and is not general-purpose, so the target interface gains a first-GPR index.

4.4 Build the concrete oracle

Workstream D — oracle backend `independent of A, B and C`

- **Derive the instruction set from the model** rather than from an external opcode list, so the oracle exercises only what this model implements.
- **Run the model and read the state back**, turning the resulting architectural state into the test's expectation.
- **Assemble and link exactly as the model does**, reusing the Golden Model's own runtime and linker script so a generated test is executable by construction.
- **Emit both output profiles** through one shared format layer, so the two backends cannot drift into mutually incompatible layouts.

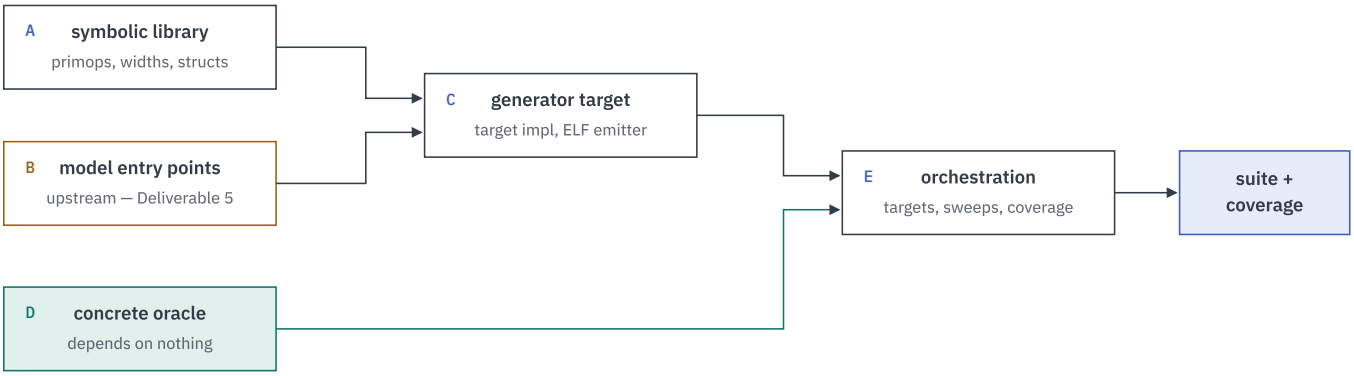
> **This workstream depends on none of the others**, which is what makes it the schedule's safety margin. It needs no solver, no IR and no upstream model change, so it can proceed in parallel with A–C and carries the floating-point and vector configurations regardless of how the symbolic path lands.

4.5 Build the driver and measurement layer

Workstream E — orchestration `written new`

- **Configuration resolution** — one validated object from which every downstream value derives, so no two components can disagree about the configuration.
- **Target enumeration and the reachable denominator** — every declared target evaluated against the configuration's predicates, partitioned into reachable and excluded with one recorded reason each, with nothing left over.
- **Routing and preamble construction** — the declarative table of section 2.2, and the builder that constructs privileged machine state directly rather than asking a solver to discover it.
- **Sweep drivers** per extension, per CSR, and per privileged scenario — the last of these because roughly ten privileged extensions define no mnemonics at all and exist only as CSR addresses.
- **Coverage collection and reconciliation**, per test and per suite, against the model's own span manifest.
- **Reporting** — per-extension status split by cause, regression against a stored baseline, and the classified uncovered-target list that closes the loop.

4.6 Dependencies between workstreams



Critical path: A → C → E. B's *merge*, not its engineering, is the least controllable element.

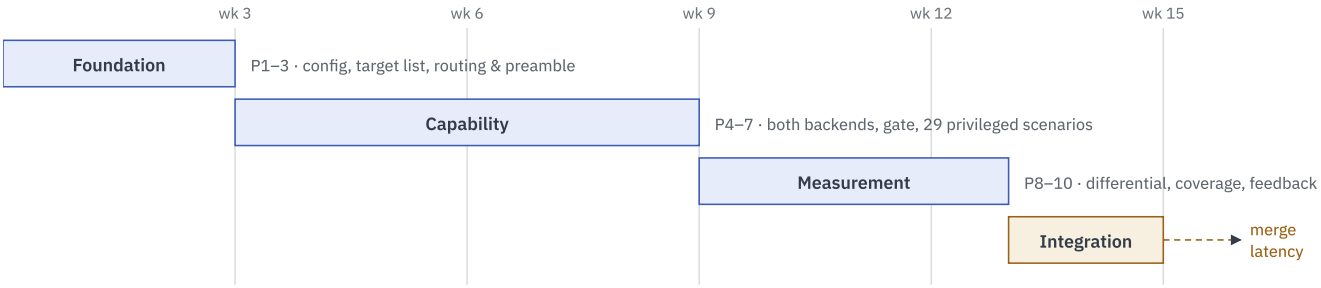
A, B and D proceed concurrently. D is the schedule's safety margin: it needs no solver, no IR and no upstream change, so it carries the floating-point and vector configurations regardless of how the symbolic path lands.

WORKSTREAM	DEPENDS ON	BLOCKS
A — symbolic library	—	C, and all symbolic generation
B — model entry points	—	C; upstream merge gates Deliverable 5
C — generator target	A and B	Symbolic-routed targets only
D — concrete oracle	—	—
E — orchestration	C or D for real output	All measurement and reporting

A, B and D can proceed concurrently. The critical path runs A → C, with B's *merge* — not its engineering — as the schedule's least controllable element.

4.7 Schedule

One number, not a range, so we can be held to it. It assumes AI-assisted delivery. That is realistic where a phase is well-scoped and mechanical — parsing, list-building, routing, report generation — and most of this roadmap is.



Track 4's two weeks are engineering effort. Completion depends on a review cadence this project does not set.

Track 2 is the longest because Phase 7 is twenty-nine individual privileged scenarios, not one mechanism applied once.

TRACK	PHASES	DELIVERS	WEEKS
1 — Foundation	1–3	Configuration resolution; reachable target list; routing and preamble builder	3
2 — Capability	4–7	Both generation paths; validation gate; privileged scenario harnesses	6
3 — Measurement	8–10	Differential execution; coverage collection and analysis; feedback interface	4

TRACK	PHASES	DELIVERS	WEEKS
4 — Integration	11–12	Golden Model CI integration; upstream submission of model changes	2

Track 2 is the longest because Phase 7 is twenty-nine individual privileged scenarios, not one mechanism applied once. Track 4's two weeks are *engineering effort*, not time to completion: both phases are done only when merged, on a review cadence this project does not set.

Milestones: what each phase builds, and when

Each phase names the workstream it advances, the tasks it covers, and one acceptance test. We state the test as something observable. A phase is done when its evidence exists, not when its effort is spent.

PH	WS	IMPLEMENTATION TASKS	WKS	ACCEPTED WHEN
1	**E B**	Configuration resolution — one validated object from which every downstream value derives. Entry points proposed upstream	1	A configuration resolves to one validated object and the correct IR, for every required XLEN
2	**E**	Target enumeration; reachable and exclusion lists built from the model's declarations	1	Every declared target partitions into reachable or excluded, with nothing left over
3	**E**	Routing table; preamble builder for privileged and CSR-only targets	1	Every reachable target routes to exactly one backend; privileged state builds from the preamble alone
4	**A C**	Engine primitives the model calls; address-width fix; struct-typed vector elements. RISC-V target implementation and the three portability fixes	1	The engine derives operands for the targeted path set on every symbolic-routed extension
5	**D**	Model-derived instruction set; run-and-capture; toolchain assembly and linking	1	The oracle generates a passing, checkable test for every oracle-routed target
6	**C E**	Validation gate; materialisation through one shared format layer; both output profiles	1	No test with an empty expected result reaches the suite; every trap scenario carries a negative control that fails
7	**E B**	Privileged scenario harnesses — PMP, translation, trap, interrupt, delegation — on the preamble builder, with the model-side PMP reset fix	3	All 29 privileged extensions and capabilities have at least one generated test
8	**E**	Differential execution across Sail, Spike, QEMU and RTL; verdict comparison and triage	2	Every materialised test runs on Sail and at least one independent simulator, with a recorded verdict
9	**E**	Instrumented build, per-test isolation, accumulation and reconciliation against the span manifest	1	Every span reconciles to hit, gap, or exclusion; nothing unaccounted for
10	**E**	Uncovered-target classification; the machine-readable feedback interface; suite and regression reporting	1	The uncovered-target list is published, machine-readable, classified by cause
11	**E**	The two CI jobs of section 2.6, and the checked-in suite they replay (D4)	1	The framework's PR is **merged** into the Golden Model's CI, not merely approved
12	**B**	Upstream submission of the model changes of section 4.2 (D5)	1	Every required change is **merged** upstream, not merely filed

> **Phases 11 and 12 are the only two whose acceptance this project cannot unilaterally satisfy.** Their engineering is a week each; their completion waits on a review cadence the Golden Model's maintainers set. That is why the critical path runs through merge latency rather than through effort.

Cost basis

Cost comes from the phase table above: **15 engineering-weeks**, at an agreed rate. Rate and team composition are commercial terms, agreed separately. Once set, the same breakdown drives both the schedule and the price.

BASIS	TREATMENT
Engineering effort	15 weeks, derived per phase. No phase is costed by guesswork, and none is skipped to reach a lower number
Upstream merge latency	**Tracked, not priced.** It is calendar time, not engineering effort; this plan does not charge for waiting on someone else's review
Delivery model	Human-plus-AI-assisted. Every upstream contribution is authored and reviewed by a named engineer who is accountable for it — see section 6
Rate and team	Commercial terms, agreed separately. Stated here as a dependency rather than a number, so the technical plan can be evaluated on its own

5 · RISK

What could go wrong, and what happens then

RISK	RESPONSE
Upstream merge latency	Engage maintainers early rather than at submission. PR state is reported as what it is — created, reviewed, approved — never as complete
Model entry points rejected	Fall back to oracle-only generation; privileged scenarios move to preamble construction. A capability reduction, not a project failure
Representational ceiling	Targets beyond the engine's 129-bit value bound route to the oracle automatically, recorded as an architectural reason. Routing is per target, so a ceiling on one extension never blocks generation elsewhere
A further missing primitive found late	Phase 4's acceptance test is a clean symbolic run over *every* symbolic-routed extension, not a sample, so the discovery happens inside the phase that owns it
Tests that cannot fail	The validation gate, mandatory negative controls, and differential execution — section 2.4

6 · LICENSING AND CONTRIBUTION

How the licences compose

We license the framework **Apache-2.0**, which the RFP names as preferred. The Golden Model is **BSD-2-Clause**. Both are permissive and compatible in this direction. Framework code that calls into or ships alongside the model puts no copyleft obligation back on the model, so integrating into the Golden Model's CI needs no relicensing either way.

Third-party components

COMPONENT	LICENCE	ROLE
sail-riscv (Golden Model)	BSD-2-Clause	The specification under test; consumed unmodified
isla, isla-sail, isla-testgen	BSD-2-Clause	Symbolic execution engine and IR compiler
Z3	MIT	SMT solver, reached only through isla
Spike (riscv-isa-sim)	BSD-3-Clause	Independent execution target
QEMU	GPL-2.0	Independent execution target

COMPONENT	LICENCE	ROLE
CVA6	Solderpad v2.0	RTL execution target
Verilator	LGPL-3.0 / Artistic-2.0	RTL simulator
RISC-V GNU toolchain	GPL-3.0 with runtime exception	Assembler and linker
riscv-arch-test	Apache-2.0 / BSD / CC	Compatibility target

> **The copyleft tools put no obligation on us.** We invoke QEMU, Verilator and the GNU toolchain as separate executables — an assembler is run on a file, a simulator is run on an ELF and its exit code is read. We do not link against, embed, or derive from any of them. The GPL attaches to distributing derived works, not to being executed as a tool, so the framework's Apache-2.0 licence is unaffected and no distributed artefact inherits copyleft. The generated tests are likewise unaffected: they are outputs produced from the Sail model's own declarations, not derivatives of any tool that runs them. CVA6's Solderpad licence — Apache-2.0 with hardware-specific terms — is covered by the same reasoning, since it is a target the ELFs run on rather than a component the framework distributes.

Upstream submissions

Anything we submit into the Sail model itself — the model changes of Deliverable 5, and the entry points of section 4.2 — becomes part of sail-riscv. We contribute it under **sail-riscv's own licence and contribution terms**, not ours. Its `CONTRIBUTING.md` governs, and we check it at submission time rather than assume it here.

The schedule assumes AI-assisted delivery, so we state the contribution terms rather than leave a reviewer to infer them. A named engineer authors and reviews every change we submit upstream, and signs it off under sail-riscv's terms, including a DCO sign-off where required. AI assistance does not change who the contributor of record is. We submit nothing a named engineer has not reviewed and is not accountable for.

7 · MAINTENANCE

After delivery

We deliver the framework as its own repository under an RVI-designated organisation, and transfer it with its history intact. We agree ownership — admin, review rights, release authority — at project start, not at delivery.

Developer documentation covers what someone extending the framework cannot infer: how we classify a new extension into the routing table and what evidence that needs, how to add a backend behind the same interface, and how the target list derives from the model's declarations so it keeps working as the model changes. The model and engine revisions the framework was verified against will be pinned as data, with a documented bump procedure that re-runs generation and compares against the stored baseline — so a pin moves only when the evidence for the new pin exists.

> **We record why each decision was made.** Why a backend is routed one way, why an exclusion is functional rather than scheduling, why a convention exists. A maintainer who cannot reconstruct why a constraint exists will eventually remove it — which is the failure mode this is written to prevent.

Every figure and table in this document is derived from the Sail model's own declarations or from the RFP. Quantities attributed to the model describe the problem to be solved, not work already performed, and will be recomputed against the model revision agreed at project start.